指令

数据表示

- 数据表示与数据类型
  - **数据表示是**计算机硬件能**直接**识别的，可以被指令系统**直接**调用的**数据类型**
  - 数据结构-->软件映像-->数据表示
  - 要确定那些数据类型使用数据表示来实现，随着计算机系统的发展，**一些复杂的数据表示**在某些计算机系统中出现
- 高级数据表示
  - 一般计算机中数据的属性**通过**指令中的**操作码解释**
    - 数据类型
    - 寻址方式等
  - 为了弥补语义差距可以让机器中每个数据都加上类型标志位 — 对程序员透明 — 数据存储量增加，指令存储量减少
  - 采用标识符数据表示方法
    - 简化指令系统，程序设计，编译器等
    - 缺点：数据和指令的长度不一致，指令执行速度降低，硬件复杂
  - 采用数据描述符 — 标识符作用与一个数据，数据描述符作用于一组数据
  - 可以增设 数组，向量的表示 为快速运算**提供更好的硬件支持** — 组成 向量机
  - 堆栈
    - 通用寄存器型机器对堆栈数据结构实现的支持较差
    - 堆栈机器的特点
      - 高速寄存器组
      - 有堆栈指令
      - 高级语言汇编
      - 子程序嵌套
- 引入数据表示的原则 — 效率是否能提高，存储是否能减少
- 浮点数表示方法 — 浮点数表示
  - 尾数
  - 阶码
  - 尾数基值
  - 阶码基值
  - 尾数长度
  - 阶码长度
  - 尾数决定 浮点数的**精度**
  - 阶码决定浮点数的**表示范围**
  - 基数越大，相同阶码能表示的数的范围增加，对阶更快 — 精度损失
  - **非负阶、规格化、正尾数**（r_m 为2的整数次幂时，$r_m^{m'}=2^m$）
    - 可表示最小尾数（小数点后第1个r_m 进制数位为1）：r_m^{-1}
    - 可表示最大尾数（r_m 进制尾数均为r_m-1）：$\sum_{i=1}^{m'} r_m^{-i}=1-r_m^{-m'}=1-2^{-m}$
    - 最大阶值（阶值部分全为1）：2^p-1
    - 可表示最小值：r_m^{-1}
    - 可表示最大值：$(1-r_m^{-m'})\times r_m^{2^p-1}=r_m^{2^p-1}\times(1-2^{-m})$
    - 可表示尾数个数：$r_m^{m'-1}(r_m-1)=2^m\times(r_m-1)/r_m$
    - 可表示阶的个数：2^p
    - 可表示数的个数：$2^p r_m^{m'-1}(r_m-1)=2^{p+m}(r_m-1)/r_m$
  - 可表示的尾数的个数
    - 对于第一位 d_1：因为不能是 0，所以只能在 $1\sim(r_m-1)$ 之间选择，共有 (r_m-1) 种选法。
    - 对于剩下的 $(m'-1)$ 位（d_2 到 $d_{m'}$）：每一位都可以是 $0\sim(r_m-1)$ 的任意数，所以每一位有 r_m 种选法，共 $r_m^{m'-1}$ 种选法。
    - 尾数一共是m'位 — 去掉第一位就是m'-1位，这里面的每一位都有基数（rm）个可选的项,即 rm^{m'-1}，第一位只有rm-1个可选的，所有的尾数的可表达个数为**(rm-1)(rm^{m'-1})**
    - m=m'log(rm),这里等式左右两边同时取2的幂 — **rm**^{m'} — 2^{m}
  - 可表示的数的个数 — 用可表示的阶的数量 * 可表示的尾数的数量
  - 观察此时能表示的最小值 rm^{-1}，最大值1-rm^{-m'}*rm^{2^p-1}
    - 随着rm的增大，**可表示的最小值减少，可表示的最大值增加， 即 表达范围变大了**
    - 可表示的数的个数，随着rm的增大，**可表示数的个数增大**
    - 数的密度 变 稀疏 — 精度下降
    - 对阶次数减少，**运算速度提高**
  - 尾数下溢的处理方法
    - 目的为了把溢出的g位数据合理地**折算回现有的p位有效尾数中，并尽量减少误差**
    - 截断
    - 舍入 — 通过附加位进行判断
      - 如果附加位为0 — 说明 丢弃的部分少于0.5
      - 如果附加位为1 — 说明 丢弃的部分大于0.5 — 尾数+1
    - 恒置1法 — 将多的部分丢弃，
      - 如果原本是0的情况 — 多了1lsb-0.5lsb — 0.5lsb
      - 如果原本是1的情况 — 置为1，因为进行舍入-0.5lsb
      - 总体来说误差为0
    - 查表法
      - 1. 硬件结构（Slide 5）
        - 把 p 位尾数切成两部分：高 $(p-n)$ 位和低 n 位。
        - 将低 n 位与下溢 g 位的最高 1 位拼接，组成一个 $(n+1)$ 位的地址，输入到一个小容量的 ROM 或 PLA（只读存储器/可编程逻辑阵列）中。
        - ROM 查表后，直接输出修正后的 n 位尾数送回。
        - 关键点：ROM 的输出只有 n 位，绝不向高 $(p-n)$ 位进位，从而彻底切断了全局进位链，保证了速度。

指令系统的发展和改进

- 两种途径
- CISC — 增强原有指令功能以设置更复杂的新指令，取代原来由软件子程序完成的功能（**原来需要多条指令才能完成的**）
  - 28定律 — 20%的指令**占用了80%处理机的执行时间**
  - 大规模集成电路技术的发展 — 70年代后使用DRAM
    - 使得主存和控存的**速度相当**
    - 简单指令没有必要使用微程序
    - 复杂指令使用微程序实现与用简单指令组成子程序时间没有区别
  - 软硬件分配问题
- RISC
  - **减少**指令种类和**简化**指令功能来**降低**硬件设计的**复杂度**
    - 减少指令和寻址方式种类
    - 固定指令格式
    - 大多数指令在单周期内完成
    - 再用Load/Store结构 扩大通用寄存器的数量
    - 硬布线逻辑
    - 简化编译
  - 减少CPI — 即 让**一条指令执行的更快**
- RICS关键技术
  - 采用高速缓存Cache，设置指令cache和数据cache分别存放
  - 优化设计编译系统
  - 延时转移技术
    - 在流水线中 分支指令需要经历 译码，条件判断等阶段 传统做法是 流水线stall
    - RISC采用**分支延迟槽** — **将无关指令**挪到JMP指令后面
  - 指令取消技术 — 当找不到合适的指令填补延迟槽的时候，RISC**采用指令取消技术**， 允许编译器冒险将一条可能用到的指令**塞进延迟槽**， 猜错了由硬件取消
  - 重叠窗口技术
    - 传统CICS架构中，函数A调用函B的时候，需要**把参数压入内存，B从内存中读出来**
    - 解决方式
      - 重叠窗口
      - 不使用内存传参，使用寄存器堆
        - 例如
          - 全局寄存器
          - 局部寄存器
          - 输入寄存器 — 接受上层调用者的参数
          - 输出寄存器 — 存放下层被调用者的参数
        - 如果A调用B的话，A的**输出**寄存器与B的**输入**寄存器，**实际上是同一个,降低开销**

寻址方式

- 编址方式
  - 编址单位
    - 字节编址 — 存放策略
      - 策略 (c)：从地址的整倍数位置开始访问（边界对齐 / 自然对齐 - 第 5 张幻灯片）
      - 这是现代计算机系统（包括 C 语言编译器）默认采用的工业标准。
        - 设计思想（自然对齐规则）：
          - 数据的存放起始地址，必须是该数据类型自身大小的整数倍。
          - 双字（8 字节）：起始地址必须能被 8 整除（二进制地址末尾 3 位必须是 000）。
          - 单字（4 字节）：起始地址必须能被 4 整除（二进制地址末尾 2 位必须是 00）。
          - 半字（2 字节）：起始地址必须能被 2 整除（二进制地址末尾 1 位必须是 0）。
          - 字节（1 字节）：可以存放在任意地址。
        - 为什么能保证"一个周期内完成"？
          - 因为只要遵守这个规则，任何数据都绝对不可能跨越物理存储字（8字节）的边界。
          - 例如：一个 4 字节的单字如果从地址 4 开始存，它占用字节 4, 5, 6, 7，刚好在一个字内；如果从地址 6 开始存（不合规），它就会占用字节 6, 7 和下一个字的 0, 1，发生跨界。符合对齐规则就保证了 1 次访存必定成功。
        - 折中表现：
          - 空间浪费适度：只在不同大小的数据交替存放时，产生少量的填充空隙（Padding）。
          - 控制逻辑中等：虽然仍需要多路选择器来选择字内的不同字节，但避免了最复杂的"跨字拼接"逻辑。
    - 字编址
    - 位编址
    - 块编址
  - 地址空间
    - 专题一：零地址空间个数（Slide 1）
      - 在计算机中，"零地址空间"指的是独立从地址 0 开始编址的存储区域。根据系统设计，这些空间可以独立存在，也可以合并。
      1. 三个零地址空间（独立编址）：
         - 空间 1：通用寄存器（如 R0, R1...）
         - 空间 2：主存储器（内存）
         - 空间 3：输入输出设备（I/O 端口）
         - 特点：这三个空间都从 0 开始编址。CPU 必须使用不同的指令来区分它们（例如：用 ADD 访问寄存器，用 LD/ST 访问内存，用 IN/OUT 访问 I/O）。
      2. 两个零地址空间（统一编址/内存映射 I/O）：
         - 空间 1：通用寄存器。
         - 空间 2：主存储器与 I/O 设备合并。I/O 寄存器被分配在内存地址空间的某一段（通常是最高端）。
         - 特点：CPU 可以使用同一套内存读写指令（如 MOV）来访问内存和 I/O 设备，硬件设计更简单。
      3. 一个零地址空间：
         - 将寄存器、内存、I/O 全部塞进一个单一的线性地址空间。最底端是寄存器，中间是内存，最高端是 I/O。
      4. 隐含编址方式：
         - 像堆栈（通过栈顶指针 SP 隐含访问）和 Cache（由硬件自动寻址，对程序员透明），它们不需要在指令中显式给出地址。
    - 交叉编址
      - 低位交叉 — 提高速度
      - 高位交叉 — 增加容量
    - 设备编址
      - 一台设备一个地址
      - 一台设备两个地址
      - 多个编址寄存器共用同一个地址
      - 设备中每个寄存器都有自己的地址
- 寻址方式
  - 立即数寻址 — 指令中给出**操作数**
  - 面向寄存器的寻址方式 — 在指令的执行过程中需要的操作数**来自寄存器**， 运算结果**也写回寄存器**
  - 面向主存储器的寻址方式
    - 直接寻址 — 给出 操作数 或者 运算结果 **所存放的主存地址**
    - 间接寻址 — 经过两次访问主存的操作才能得到操作数
    - 变址寻址 — 使用**变址寄存器**给出**基地址**+指令给出**偏移量** 得到**有效地址**
  - 面向堆栈的寻址方式 — 此时 地址是隐含的，不需要在指令中给出
- 定位方式 — 研究程序的主存物理地址在**什么时间确定**，采用**什么方式来实现**
  - 直接定位方式 — 装入前确定
  - 静态定位
  - 动态定位
    - 增加相应的**基址寄存器**和地址加法器件，将装入主存的起始地址**存入对应该道程序的基址寄存器中**
    - 程序执行时通过地址加法器将逻辑地址+基址寄存器的程序基址，**形成有效地址**
    - 可在指令中添加相应的标志位来**指明指令地址是否需要**加基址

指令系统的优化设计

- 指令组成
  - 目的
    - 节约程序的存储空间
    - 格式规整，**便于译码**
  - 操作码+地址码
  - 地址码通常包含三部分内容
    - 地址
    - 地址附加信息
    - 寻址方式
  - 操作码
    - 操作种类
    - 操作数描述
    - 长度
      - 固定长度
      - 扩展编码 — 不同的扩展方式
        - 1. 左表：等长 15/15/15 扩展法
          - 这种方案的特点是：每一级都只留 1 个编码作为下级的"扩展前缀"，从而让本级可用的指令数最大化。
          - 第 1 级（4位操作码）：
            - 4 位总共可以组合出 $2^4=16$ 种状态。
            - 我们使用其中的 15 种（0000 → 1110）作为 4 位指令。
            - 剩下 1 种（1111）作为扩展标志（前缀），不分配给具体指令。
          - 第 2 级（8位操作码）：
            - 前缀固定为 1111（4位），后面新增 4 位，可组合出 $2^4=16$ 种状态。
            - 我们使用其中的 15 种（1111 0000 → 1111 1110）作为 8 位指令。
            - 剩下 1 种（1111 1111）继续作为扩展前缀。
          - 第 3 级（12位操作码）：
            - 前缀固定为 1111 1111（8位），后面新增 4 位，可组合出 $2^4=16$ 种状态。
            - 使用其中的 15 种（1111 1111 0000 → 1111 1111 1110）作为 12 位指令。
        - 2. 右表：等长 8/64/512 扩展法
          - 这种方案的特点是：在早期留下更多的"扩展前缀"，从而让后期（长操作码）的指令数量呈指数级爆发。
          - 第 1 级（4位操作码）：
            - 共 16 种状态，我们只使用 8 种（0000 → 0111）。
            - 剩下 8 种（1000 → 1111）全部用来作为扩展前缀。
          - 第 2 级（8位操作码）：
            - 每一个前缀都可以向后扩展 4 位，因此总共有 $8\times2^4=128$ 种状态。
            - 我们使用其中的 64 种。
            - 剩下 $128-64=64$ 种作为下一级的扩展前缀。
          - 第 3 级（12位操作码）：
            - 每一个前缀向后扩展 4 位，总共有 $64\times2^4=1024$ 种状态。
            - 我们使用其中的 512 种。
        - 第二张图：不等长操作码扩展法（4-6-10 扩展）
          - 当地址字段的长度不均匀时，操作码扩展的步长也是不均匀的。我们以表中 8/16/256 这一例，推导它是如何计算的：
          - 第 1 级（4位操作码）：
            - 总状态数：$2^4=16$。
            - 使用 8 条指令。
            - 留给下一级扩展的码数 = $16-8=8$ 个。
          - 第 2 级（6位操作码，增加了 2 位）：
            - 上一级留下的 8 个前缀，每个扩展 2 位，可产生：$8\times2^2=32$ 个状态。
            - 使用 16 条指令。
            - 留给下一级扩展的码数 = $32-16=16$ 个。
          - 第 3 级（10位操作码，增加了 4 位）：
            - 上一级留下的 16 个前缀，每个扩展 4 位，可产生：$16\times2^4=256$ 个状态。
            - 使用 256 条指令。
          - 总指令种类 = $8+16+256=280$ 种。
          - *(你可以用同样的方法验算最后一行的 4/32/256，其总指令数为 292 种。)*
      - Huffman编码 — 将节点按照频率从小到大排列，然后每次选择两个**频率最小的合成新节点**， 新节点也参与选择的过程，如此反复
- 操作码优化设计 — 拓展编码例题

若某机有要求：三地址指令4条，单地址指令255条，零地址指令16条，设指令字长为12位，每个地址码长为3位。能否以扩展操作码为其编码？如果单地址指令改为254条呢？

- 三地址指令操作码为3位，可表示8个码，其中4个码（如000、001、010、011）用来表示4条三地址指令，剩余4个码（如100、101、110、111）用来扩展。
- 若操作码9位，利用剩余4个扩展码可以最多可以表示256个操作码。
- 若单地址指令用掉其中255个码，则无法表示剩余的16条零地址指令。

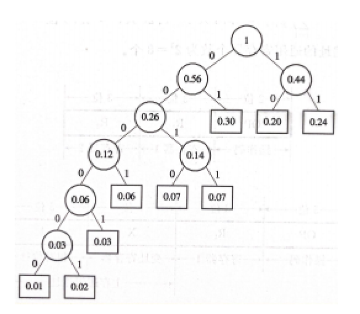
操作码（3位）	地址1（3位）	地址2（3位）	地址3（3位）
操作码（9位）			地址3（3位）
操作码（12位）			

- 指令字格式优化设计
  - 优化途径
    - 采用拓展码
    - 采用多种寻址方式
    - 多地址制
    - 同种地址制内采用多种地址形式
    - 使用多种不同指令字长度
  - 例题

某模型机9条指令的使用频度如下表所示。要求有两种指令字长，都按双操作数指令格式编排，采用扩展操作码，并限制只能有两种操作码码长。设该机有若干通用寄存器，主存为16位宽，按字节编址，采用按整数边界存储，任何指令都在一个主存周期中取得，短指令为寄存器-寄存器型，长指令为寄存器-主存型，主存地址应能变址寻址。

指令	频度	指令	频度	指令	频度
ADD（加）	30%	SUB（减）	24%	JOM（按负转移）	6%
STO（存）	7%	JMP（转移）	7%	SHR（右移）	2%
CIL（循环左移）	3%	CLA（清加）	20%	STP（停机）	1%

97



98

指令	频度	哈夫曼编码	扩展编码
ADD（加）	30%	01	00
SUB（减）	24%	11	01
CLA（清加）	20%	10	10
JOM（按负转移）	6%	0001	11000
STO（存）	7%	0011	11001
JMP（转移）	7%	0010	11010
SHR（右移）	2%	000001	11011
CIL（循环左移）	3%	00001	11100
STP（停机）	1%	000000	11101

- 哈夫曼编码平均码长：2. 61
- 2-5扩展编码平均码长：2. 78

操作码（2位）	寄存器1（3位）	寄存器1（3位）

操作码（5位）	寄存器1（3位）	变址寄存器（3位）	相对位移（5位）

  - 基本同理的例题

2. 一台模型机共有 7 条指令，各指令的使用频度分别为 35%、25%、20%、10%、5%、3%、2%，有 8 个通用数据寄存器，2 个变址寄存器。
   1) 要求操作码的平均长度最短，请设计操作码的编码，并计算所设计操作码的平均长度
   2) 设计 8 位字长的寄存器-寄存器型指令 3 条、16 位字长的寄存器-存储器型变址寻址方式指令 4 条，变址范围不小于正、负 127。请设计指令格式，并给出各字段的长度和操作码的编码。

Presented with xmind